

PROGRAM 5

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, davies_bouldin_score
from sklearn.decomposition import PCA

# ----- Load and Prepare Data -----
# Load dataset
data = pd.read_csv("heart.csv")

# Drop the target column if it exists
if 'target' in data.columns:
    data = data.drop(columns=['target'])

# Keep only numeric columns
data = data.select_dtypes(include=['int64', 'float64'])

# Drop missing values
data = data.dropna()

# Normalize data
scaler = StandardScaler()
data_scaled = scaler.fit_transform(data)

# ----- PCA for Visualization -----
pca = PCA(n_components=2)
data_pca = pca.fit_transform(data_scaled)

# ----- K-Means Clustering -----
k = 3 # Choose number of clusters
kmeans = KMeans(n_clusters=k, random_state=42)
kmeans_labels = kmeans.fit_predict(data_scaled)

# Evaluate K-Means
kmeans_silhouette = silhouette_score(data_scaled, kmeans_labels)
kmeans_db = davies_bouldin_score(data_scaled, kmeans_labels)

# ----- GMM Clustering (EM Algorithm) -----
gmm = GaussianMixture(n_components=k, covariance_type='full', random_state=42)
gmm_labels = gmm.fit_predict(data_scaled)

# Evaluate GMM
```

```

gmm_silhouette = silhouette_score(data_scaled, gmm_labels)
gmm_db = davies_bouldin_score(data_scaled, gmm_labels)

# ----- Visualization -----
plt.figure(figsize=(12, 5))

# Plot K-Means clusters
plt.subplot(1, 2, 1)
plt.scatter(data_pca[:, 0], data_pca[:, 1], c=kmeans_labels)
plt.title("K-Means Clustering")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")

# Plot GMM Clusters
plt.subplot(1, 2, 2)
plt.scatter(data_pca[:, 0], data_pca[:, 1], c=gmm_labels)
plt.title("GMM (EM Algorithm) Clustering")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")

plt.tight_layout()
plt.show()

# ----- Print Results -----
print("\n=== Clustering Evaluation ===")
print(f"K-Means Silhouette Score: {kmeans_silhouette:.4f}")
print(f"GMM (EM) Silhouette Score: {gmm_silhouette:.4f}")
print(f"K-Means Davies-Bouldin Index: {kmeans_db:.4f}")
print(f"GMM (EM) Davies-Bouldin Index: {gmm_db:.4f}")

print("\n=== Conclusion ===")
if gmm_silhouette > kmeans_silhouette:
    print("✅ GMM (EM) gives better cluster separation based on Silhouette Score.")
else:
    print("✅ K-Means gives better cluster separation based on Silhouette Score.")

if gmm_db < kmeans_db:
    print("✅ GMM (EM) forms more compact clusters based on Davies-Bouldin Index.")
else:
    print("✅ K-Means forms more compact clusters based on Davies-Bouldin Index.")

```

 `import pandas as pd`

This imports the **pandas** library and gives it the nickname `pd`. Pandas is essential for working with structured data like CSV files. It lets you load, manipulate, and analyze tabular data easily — think of it as a powerful spreadsheet tool inside Python.

 `import matplotlib.pyplot as plt`

This brings in **matplotlib's pyplot module**, which is used for creating visualizations like line charts, scatter plots, and bar graphs. The nickname `plt` is standard and makes plotting commands shorter and cleaner.

 `from sklearn.cluster import KMeans`

This imports the **KMeans clustering algorithm** from scikit-learn. KMeans is a popular unsupervised learning method that groups data into clusters based on similarity. You specify how many clusters you want, and it tries to find the best grouping.

 `from sklearn.mixture import GaussianMixture`

This imports the **Gaussian Mixture Model (GMM)**, which is another clustering method. Unlike KMeans, which assigns each point to one cluster, GMM uses probabilities — meaning a point can belong to multiple clusters with different likelihoods. It's more flexible and often better when clusters overlap.

 `from sklearn.preprocessing import StandardScaler`

This brings in **StandardScaler**, which standardizes your data. It transforms each feature so that it has a mean of 0 and a standard deviation of 1. This is crucial because clustering algorithms are sensitive to scale — without this, features with larger ranges could dominate the clustering.

 `from sklearn.metrics import silhouette_score,
davies_bouldin_score`

These are **evaluation metrics** for clustering:

- `silhouette_score` tells you how well-separated your clusters are. It ranges from -1 to 1, and higher is better.
- `davies_bouldin_score` measures how similar clusters are to each other — lower values are better, meaning clusters are compact and distinct.

Using both gives you a more complete picture of clustering quality.

 `from sklearn.decomposition import PCA`

This imports **Principal Component Analysis**, a technique used to reduce the number of features (dimensions) in your dataset. It helps you visualize high-dimensional data in 2D or 3D plots while preserving the structure. You'll often use PCA before plotting clusters to make them easier to see.

Loading the Dataset

```
python
data = pd.read_csv("heart.csv")
```

This line loads the `heart.csv` file into a pandas DataFrame named `data`. A DataFrame is like a table with rows and columns, and it's the standard way to work with structured data in Python. This step is essential because it gives you access to the dataset so you can clean, transform, and analyze it.

Dropping the Target Column

```
python
if 'target' in data.columns:
    data = data.drop(columns=['target'])
```

Here, you're checking if the dataset contains a column named `'target'`, which usually represents the label or outcome in supervised learning (e.g., whether someone has heart disease). Since clustering is an **unsupervised** technique, you don't want to include this column — it would bias the clustering. So, if it exists, you drop it.

Keeping Only Numeric Columns

```
python
data = data.select_dtypes(include=['int64', 'float64'])
```

Clustering algorithms like k-Means and GMM require **numerical input**. This line filters the DataFrame to keep only columns with integer (`int64`) or floating-point (`float64`) data types. It automatically excludes categorical or text columns that can't be used directly in clustering.

Handling Missing Data

```
python
data = data.dropna()
```

Missing values can cause errors or distort clustering results. This line removes any rows that contain `NaN` (missing) values. It's a quick way to clean the dataset, though in practice you might consider imputing missing values instead of dropping them, especially if the dataset is small.

Normalizing the Data

```
scaler = StandardScaler()
data_scaled = scaler.fit_transform(data)
```

This step standardizes the numeric features so that each one has a mean of 0 and a standard deviation of 1. It's crucial because clustering algorithms are sensitive to feature scales — for example, a feature like age (ranging from 20–80) could dominate another feature like cholesterol (say, 100–300) if not scaled. `StandardScaler` ensures all features contribute equally to the clustering process.

Dimensionality Reduction with PCA

```
python
pca = PCA(n_components=2)
data_pca = pca.fit_transform(data_scaled)
```

This part uses **Principal Component Analysis (PCA)** to reduce the number of features in your dataset to just two. Why two? Because it makes it easier to **visualize** the data in a 2D scatter plot later. PCA works by finding the directions (called principal components) that capture the most variance in the data. Even though your original dataset might have many features, PCA compresses it while preserving the most important patterns. The result, `data_pca`, is a simplified version of your dataset with just two columns.

K-Means Clustering

```
python
k = 3 # Choose number of clusters
kmeans = KMeans(n_clusters=k, random_state=42)
kmeans_labels = kmeans.fit_predict(data_scaled)
```

This section applies the **k-Means clustering algorithm**. You've chosen to divide the data into `k = 3` clusters. The `random_state=42` ensures that the results are reproducible — it fixes the randomness so you get the same clusters every time you run the code. The method `fit_predict` does two things: it fits the model to your scaled data and assigns each data point to one of the three clusters. The result, `kmeans_labels`, is a list of cluster assignments for each data point.

Evaluating k-Means Clustering

```
python
kmeans_silhouette = silhouette_score(data_scaled, kmeans_labels)
kmeans_db = davies_bouldin_score(data_scaled, kmeans_labels)
```

After clustering, you evaluate how well it performed using two metrics:

- **Silhouette Score:** This measures how well each point fits within its cluster compared to other clusters. It ranges from -1 to 1, and higher values mean better clustering.
- **Davies-Bouldin Score:** This measures the average similarity between clusters. Lower values are better, indicating that clusters are compact and well-separated.

These scores help you judge whether k-Means is doing a good job or if another method might be better.



GMM Clustering (Expectation-Maximization)

python

```
gmm = GaussianMixture(n_components=k, covariance_type='full', random_state=42)  
gmm_labels = gmm.fit_predict(data_scaled)
```

This part uses the **Gaussian Mixture Model (GMM)**, which is a more flexible clustering method. Instead of assigning each point to a single cluster like k-Means, GMM calculates the probability that a point belongs to each cluster. You still choose `k = 3` clusters, and `covariance_type='full'` means each cluster can have its own shape and orientation. The `fit_predict` method assigns each point to the cluster with the highest probability.



Evaluating GMM Clustering

python

```
gmm_silhouette = silhouette_score(data_scaled, gmm_labels)  
gmm_db = davies_bouldin_score(data_scaled, gmm_labels)
```

Just like with k-Means, you evaluate GMM using the **silhouette score** and **Davies-Bouldin score**. Comparing these scores with those from k-Means helps you decide which clustering method works better for your dataset.



Visualizing Clusters with PCA

python

```
plt.figure(figsize=(12, 5))
```

This line sets up a new figure for plotting, with a width of 12 inches and height of 5 inches. It creates space for two side-by-side subplots — one for k-Means and one for GMM — so you can visually compare how each algorithm grouped the data.



Plotting k-Means Clusters

python

```
plt.subplot(1, 2, 1)  
plt.scatter(data_pca[:, 0], data_pca[:, 1], c=kmeans_labels)  
plt.title("K-Means Clustering")  
plt.xlabel("PCA Component 1")  
plt.ylabel("PCA Component 2")
```

This block creates the first subplot (1 row, 2 columns, position 1) and plots the k-Means clustering results. You're using the PCA-reduced data (`data_pca`) so that each point is shown in 2D. The `c=kmeans_labels` argument colors each point based on its cluster assignment. The axis labels indicate that you're plotting the first and second principal components — these are the directions of maximum variance in the data.

Plotting GMM Clusters

```
python
plt.subplot(1, 2, 2)
plt.scatter(data_pca[:, 0], data_pca[:, 1], c=gmm_labels)
plt.title("GMM (EM Algorithm) Clustering")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
```

This creates the second subplot and plots the clustering results from the Gaussian Mixture Model (GMM). Again, you're using PCA-reduced data and coloring points by their cluster assignments. This lets you visually compare how GMM and k-Means differ in how they group the same data.

Finalizing the Layout

```
python
plt.tight_layout()
plt.show()
```

`plt.tight_layout()` adjusts the spacing between subplots so they don't overlap or look cramped. `plt.show()` displays the figure — this is the moment when your two clustering plots appear side by side.

Printing Evaluation Metrics

```
python
print("\n=== Clustering Evaluation ===")
print(f"K-Means Silhouette Score: {kmeans_silhouette:.4f}")
print(f"GMM (EM) Silhouette Score: {gmm_silhouette:.4f}")
print(f"K-Means Davies-Bouldin Index: {kmeans_db:.4f}")
print(f"GMM (EM) Davies-Bouldin Index: {gmm_db:.4f}")
```

This section prints out the evaluation scores for both clustering methods. You're showing the **Silhouette Score** (higher is better) and the **Davies-Bouldin Index** (lower is better) for each. These metrics help you assess how well the clusters are separated and how compact they are.

✓ Drawing a Conclusion

```
python
print("\n=== Conclusion ===")
if gmm_silhouette > kmeans_silhouette:
    print("✓ GMM (EM) gives better cluster separation based on Silhouette Score.")
else:
    print("✓ K-Means gives better cluster separation based on Silhouette Score.")

if gmm_db < kmeans_db:
    print("✓ GMM (EM) forms more compact clusters based on Davies-Bouldin Index.")
else:
    print("✓ K-Means forms more compact clusters based on Davies-Bouldin Index.")
```

This final block compares the scores and prints a conclusion. It uses simple `if` conditions to decide which algorithm performed better based on each metric. The use of ✓ makes the output more readable and friendly. This is a great way to summarize your findings and communicate which clustering method worked best for your dataset.

```
import pandas as pd
```

```
dataset=pd.read_csv('heart.csv')  
print(dataset.head())  
print(dataset.info())
```

```
import pandas as pd  
from pgmpy.models import DiscreteBayesianNetwork  
from pgmpy.estimators import HillClimbSearch, BIC, BayesianEstimator
```

```
from pgmpy.inference import VariableElimination
```

```
# Load dataset
```

```
data = pd.read_csv('heart.csv')
```

```
# Discretize continuous variables using pandas qcut()
```

```
def discretize_quantile(series, q, labels):
```

```
    return pd.qcut(series, q=q, labels=labels, duplicates='drop')
```

```
# Apply quantile-based binning
```

```
data['age_cat'] = discretize_quantile(data['age'], q=3, labels=['young', 'middle', 'old'])
```

```
data['trestbps_cat'] = discretize_quantile(data['trestbps'], q=3, labels=['low', 'normal', 'high'])
```

```
data['chol_cat'] = discretize_quantile(data['chol'], q=3, labels=['normal', 'high', 'very_high'])
```

```
data['thalach_cat'] = discretize_quantile(data['thalach'], q=3, labels=['low', 'medium', 'high'])
```

```
data['oldpeak_cat'] = discretize_quantile(data['oldpeak'], q=3, labels=['low', 'medium', 'high'])
```

```
# Select discretized + categorical columns for model
```

```
model_data = data[['age_cat', 'sex', 'cp', 'trestbps_cat', 'chol_cat', 'fbs', 'restecg',  
                  'thalach_cat', 'exang', 'oldpeak_cat', 'slope', 'ca', 'thal', 'target']]
```

```
# Drop any rows with missing values (if any)
```

```
model_data = model_data.dropna()
```

```
# Learn structure with Hill Climbing and BIC score
```

```
hc = HillClimbSearch(model_data)
```

```
best_model = hc.estimate(scoring_method=BIC(model_data))
```

```
print("Learned network edges:")
```

```
print(best_model.edges())
```

```
# Create Bayesian Network model with learned structure
```

```
model = DiscreteBayesianNetwork(best_model.edges())
```

```
# Parameter estimation with Bayesian Estimator
```

```
model.fit(model_data, estimator=BayesianEstimator, prior_type='BDeu')
```

```

# Inference object for queries
inference = VariableElimination(model)

# Example query: Probability of heart disease (target=1) given some symptoms
evidence = {
    'age_cat': 'old',
    'sex': 1,
    'cp': 3,
    'trestbps_cat': 'high',
    'chol_cat': 'high',
    'fbs': 0,
    'exang': 1,
}

query_result = inference.query(variables=['target'], evidence=evidence)
print("\nProbability of Heart Disease given evidence:")
print(query_result)

evidence = {
    'age_cat': 'old',
    'sex': 1,
    'cp': 3,
    'trestbps_cat': 'high',
    'chol_cat': 'high',
    'fbs': 0,
    'restecg': 1,
    'thalach_cat': 'medium',
    'exang': 1,
    'oldpeak_cat': 'medium',
    'slope': 2,
    'ca': 0,
    'thal': 3,
}

query_result = inference.query(variables=['target'], evidence=evidence)
print(query_result)

```

 **from pgmpy.models import DiscreteBayesianNetwork**

This line imports the **DiscreteBayesianNetwork** class from the **pgmpy.models** module. A **Bayesian Network** is a probabilistic graphical model that represents variables and their conditional dependencies using a directed acyclic graph (DAG). The "Discrete" part means all variables are assumed to take on discrete (categorical) values. This model is useful for reasoning under uncertainty, especially when you want to infer hidden causes or predict outcomes based on observed evidence.

 `from pgmpy.estimators import HillClimbSearch, BIC, BayesianEstimator`

This line brings in three important tools for **learning the structure and parameters** of a Bayesian Network from data:

- `HillClimbSearch` is a **structure learning algorithm** that tries to find the best network structure by iteratively improving a candidate graph. It's a greedy search method that adds, removes, or reverses edges to maximize a scoring function.
- `BIC` stands for **Bayesian Information Criterion**, a scoring function used to evaluate how well a network structure fits the data while penalizing complexity. It helps prevent overfitting.
- `BayesianEstimator` is used for **parameter learning** — it estimates the conditional probability tables (CPTs) for each node in the network using Bayesian methods, which can incorporate prior knowledge and handle sparse data better than maximum likelihood estimation.

 `from pgmpy.inference import VariableElimination`

This line imports the `VariableElimination` class, which is an **inference algorithm** used to answer probabilistic queries on a Bayesian Network. Once your network is built and trained, you can use this to compute the probability of certain outcomes given some evidence. It works by systematically eliminating irrelevant variables to simplify the computation.



Loading the Dataset

```
python
data = pd.read_csv('heart.csv')
```

This line loads the heart disease dataset from a CSV file into a pandas DataFrame named `data`. This dataset contains various medical attributes (like age, cholesterol, blood pressure) and a target variable indicating the presence or absence of heart disease. Loading it into a DataFrame allows you to manipulate and analyze the data easily.

```
# Discretize continuous variables using pandas qcut()
def discretize_quantile(series, q, labels):
    return pd.qcut(series, q=q, labels=labels, duplicates='drop')

# Apply quantile-based binning
data['age_cat'] = discretize_quantile(data['age'], q=3, labels=['young', 'middle', 'old'])
data['trestbps_cat'] = discretize_quantile(data['trestbps'], q=3, labels=['low', 'normal', 'high'])
data['chol_cat'] = discretize_quantile(data['chol'], q=3, labels=['normal', 'high', 'very_high'])
data['thalach_cat'] = discretize_quantile(data['thalach'], q=3, labels=['low', 'medium', 'high'])
```

```
data['oldpeak_cat'] = discretize_quantile(data['oldpeak'], q=3, labels=['low', 'medium', 'high'])
```

Use Quantile-Based Binning (e.g., Equal Frequency)

This method divides the data into bins with roughly the same number of observations. It's purely data-driven and doesn't require domain knowledge.

`pd.qcut()` automatically divides the data into **equal-sized quantile bins** (e.g., tertiles for `q=3`).

You no longer need to manually specify bin edges — this is helpful when you don't know domain-specific thresholds.

The `duplicates='drop'` argument ensures that if two quantile edges are the same (due to repeated values), it won't throw an error.

These lines apply the `discretize` function to five continuous medical features: age, resting blood pressure (`trestbps`), cholesterol (`chol`), maximum heart rate (`thalach`), and ST depression (`oldpeak`). Each feature is divided into meaningful categories based on medical thresholds. For example, age is categorized into "young", "middle", and "old", which helps the Bayesian Network reason in terms of interpretable groups rather than raw numbers.



Selecting Features for the Model

python

```
model_data = data[['age_cat', 'sex', 'cp', 'trestbps_cat', 'chol_cat', 'fbs', 'restecg', 'thalach_cat', 'exang', 'oldpeak_cat', 'slope', 'ca', 'thal', 'target']]
```

This line creates a new DataFrame called `model_data` that includes only the discretized and categorical variables needed for the Bayesian Network. It combines the newly created categorical columns (like `age_cat`, `chol_cat`) with existing categorical features (like `sex`, `cp`, `fbs`, etc.) and the target variable. This prepares the dataset for structure learning and inference, ensuring all variables are discrete and suitable for probabilistic modeling.

Dropping Missing Values

python

```
model_data = model_data.dropna()
```

Before building the Bayesian Network, you remove any rows with missing values. This is important because Bayesian Networks require complete data for both structure learning and parameter estimation. Missing values can disrupt the learning process or lead to incorrect probability estimates. While dropping rows is a quick fix, in larger projects you might consider imputation strategies to preserve more data.

Structure Learning with Hill Climbing and BIC

python

```
hc = HillClimbSearch(model_data)
best_model = hc.estimate(scoring_method=BIC(model_data))
```

This section learns the **structure** of the Bayesian Network — that is, which variables are connected and how. You use `HillClimbSearch`, a greedy algorithm that starts with an empty graph and iteratively adds, removes, or reverses edges to improve a scoring function. The scoring method here is **BIC (Bayesian Information Criterion)**, which balances model fit and complexity. A lower BIC score means a better trade-off between accuracy and simplicity. The result, `best_model`, contains the optimal set of edges (dependencies) between variables.

Printing the Learned Structure

python

```
print("Learned network edges:")
print(best_model.edges())
```

This prints the edges of the learned network — each edge represents a direct probabilistic dependency between two variables. For example, an edge from `age_cat` to `target` would suggest that age category influences the likelihood of heart disease. This gives you insight into the relationships discovered from the data.

Building and Training the Bayesian Network

python

```
model = DiscreteBayesianNetwork(best_model.edges())
model.fit(model_data, estimator=BayesianEstimator, prior_type='BDeu')
```

Here, you create a `DiscreteBayesianNetwork` using the learned edges. Then, you estimate the **parameters** — the conditional probability tables (CPTs) — using the `BayesianEstimator`. The `prior_type='BDeu'` (Bayesian Dirichlet equivalent uniform) adds a uniform prior to smooth the probabilities, which helps avoid zero-probability issues in sparse data. This step completes the training of your Bayesian Network.

Setting Up Inference

python

```
inference = VariableElimination(model)
```

Finally, you create an inference object using **Variable Elimination**, a standard algorithm for answering probabilistic queries. With this, you can ask questions like “What’s the probability of heart disease given high cholesterol and low exercise?” and get answers based on the learned model.

Making a Probabilistic Query

python

```
evidence = {
    'age_cat': 'old',
    'sex': 1,
    'cp': 3,
    'trestbps_cat': 'high',
    'chol_cat': 'high',
    'fbs': 0,
    'exang': 1,
}
query_result = inference.query(variables=['target'], evidence=evidence)
```

In this block, you're asking the Bayesian Network: “**Given this evidence, what is the probability that `target = 1` (i.e., the person has heart disease)?**”

The `evidence` dictionary contains observed values for several features:

- `'age_cat': 'old'` means the person is in the older age group.
- `'sex': 1` typically represents male.
- `'cp': 3` refers to chest pain type (likely asymptomatic).
- `'trestbps_cat': 'high'` and `'chol_cat': 'high'` indicate elevated blood pressure and cholesterol.
- `'fbs': 0` means fasting blood sugar is normal.
- `'exang': 1` means exercise-induced angina is present.

The `inference.query()` method uses **Variable Elimination** to compute the posterior probability of the `target` variable given this evidence. The result is a probability distribution over `target = 0` (no disease) and `target = 1` (disease), which you print out.

A More Detailed Query

python

```
evidence = {
    'age_cat': 'old',
    'sex': 1,
    'cp': 3,
    'trestbps_cat': 'high',
    'chol_cat': 'high',
    'fbs': 0,
    'restecg': 1,
    'thalach_cat': 'medium',
    'exang': 1,
    'oldpeak_cat': 'medium',
    'slope': 2,
    'ca': 0,
```

```

    'thal': 3,
}
query_result = inference.query(variables=['target'], evidence=evidence)

```

This second query adds more clinical features to the evidence:

- `'restecg': 1` refers to ECG results.
- `'thalach_cat': 'medium'` indicates moderate maximum heart rate.
- `'oldpeak_cat': 'medium'` reflects moderate ST depression.
- `'slope': 2`, `'ca': 0`, and `'thal': 3` are additional diagnostic indicators.

By providing more evidence, you're helping the model make a more confident prediction. The output will again be a probability distribution over `target`, showing how likely the person is to have heart disease given all these symptoms.

How to Interpret the Output

The printed result will look something like:

Code

```

+-----+-----+
| target | phi(target)|
+=====+=====+
| 0      | 0.25      |
| 1      | 0.75      |
+-----+-----+

```

This means there's a **75% probability** of heart disease (`target = 1`) given the evidence. You can use this to support diagnostic reasoning, risk assessment, or decision-making.

PROGRAM 3

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_score, recall_score,
classification_report, confusion_matrix

# Load dataset
dataset = pd.read_csv('synthetic_text_data.csv') # Ensure it has 'text' and 'label' columns
print(dataset.head(10))
print(dataset.info())

# Shuffle and split
train_data, test_data = train_test_split(dataset, test_size=0.2, random_state=42)

# Text vectorization
vectorizer = CountVectorizer(lowercase=True)
X_train = vectorizer.fit_transform(train_data['text'])
X_test = vectorizer.transform(test_data['text'])

# Labels
y_train = train_data['label']
y_test = test_data['label']

# Train Naive Bayes classifier
classifier = MultinomialNB()
classifier.fit(X_train, y_train)

# Predict
y_pred = classifier.predict(X_test)

# Evaluation
accuracy = accuracy_score(y_test, y_pred)
macro_precision = precision_score(y_test, y_pred, average='macro', zero_division=0)
macro_recall = recall_score(y_test, y_pred, average='macro', zero_division=0)

print(f"Accuracy: {accuracy:.2f}")
print(f"Macro Precision: {macro_precision:.2f}")
print(f"Macro Recall: {macro_recall:.2f}")
print("\nPer-class metrics:")
print(classification_report(y_test, y_pred, zero_division=0))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred, labels=sorted(dataset['label'].unique()))
cm_df = pd.DataFrame(cm, index=sorted(dataset['label'].unique()),
columns=sorted(dataset['label'].unique()))
```

```
print("Confusion Matrix (rows=true, columns=predicted):")
print(cm_df)
```

`import pandas as pd`

This line imports the **pandas** library, which is essential for handling structured data in Python. It allows you to load datasets (like CSV files), manipulate columns and rows, and perform data cleaning and analysis. In this context, you'll use pandas to read in your text dataset and prepare it for modeling.

`from sklearn.model_selection import train_test_split`

This imports the `train_test_split` function from scikit-learn, which is used to divide your dataset into **training** and **testing** subsets. The training set is used to build the model, while the testing set evaluates how well the model performs on unseen data. This split is crucial for validating the generalization ability of your classifier.

`from sklearn.feature_extraction.text import CountVectorizer`

This brings in `CountVectorizer`, a tool that converts raw text into a **matrix of token counts**. It transforms each sentence into a vector where each element represents the frequency of a word in the sentence. This is the first step in turning unstructured text into numerical data that machine learning models can understand.

`from sklearn.naive_bayes import MultinomialNB`

This imports the **Multinomial Naive Bayes** classifier, which is particularly well-suited for text classification tasks. It assumes that features (word counts) follow a multinomial distribution and applies Bayes' theorem to predict the most likely class for a given input. It's fast, simple, and surprisingly effective for many natural language processing tasks.

`from sklearn.metrics import accuracy_score, precision_score, recall_score, classification_report, confusion_matrix`

This line imports several **evaluation metrics** from scikit-learn:

- `accuracy_score` measures the overall percentage of correct predictions.
- `precision_score` evaluates how many predicted positives were actually correct.
- `recall_score` checks how many actual positives were correctly identified.
- `classification_report` summarizes precision, recall, and F1-score for each class.

- `confusion_matrix` shows a grid of true vs. predicted labels, helping you spot misclassifications.

Loading the Dataset

```
python
dataset = pd.read_csv('synthetic_text_data.csv')
print(dataset.head(10))
print(dataset.info())
```

This block loads a CSV file named `synthetic_text_data.csv` into a pandas DataFrame called `dataset`. The file is expected to contain two columns: `'text'`, which holds the input sentences, and `'label'`, which represents the category or class each sentence belongs to. The `head(10)` function displays the first 10 rows for a quick preview, and `info()` summarizes the dataset's structure — including column names, data types, and any missing values. This helps verify that the data is loaded correctly and is ready for processing.

Shuffling and Splitting the Data

```
python
train_data, test_data = train_test_split(dataset, test_size=0.2, random_state=42)
```

This line randomly splits the dataset into two parts: **80% for training** and **20% for testing**. The training set is used to build the model, while the test set evaluates how well the model performs on unseen data. The `random_state=42` ensures reproducibility — meaning the same split will occur every time you run the code. This is a standard practice to avoid overfitting and to assess generalization.

Text Vectorization with CountVectorizer

```
python
vectorizer = CountVectorizer(lowercase=True)
X_train = vectorizer.fit_transform(train_data['text'])
X_test = vectorizer.transform(test_data['text'])
```

This block converts raw text into numerical features using **CountVectorizer**, which builds a vocabulary from the training data and transforms each sentence into a vector of word counts. The `lowercase=True` argument ensures that all text is converted to lowercase before tokenization, reducing redundancy (e.g., "Apple" and "apple" are treated the same). `fit_transform()` is applied to the training data to learn the vocabulary and generate vectors, while `transform()` is used on the test data to apply the same vocabulary without modifying it.

Extracting Labels

```
python
```

```
y_train = train_data['label']
y_test = test_data['label']
```

Finally, this section extracts the target labels from the training and testing sets. These labels represent the correct class for each sentence and are used to train the classifier (`y_train`) and evaluate its predictions (`y_test`). This completes the preprocessing pipeline, setting the stage for model training and evaluation.

Training the Naive Bayes Classifier

```
python
classifier = MultinomialNB()
classifier.fit(X_train, y_train)
```

This block initializes and trains a **Multinomial Naive Bayes** classifier. This model is ideal for text classification tasks where features represent word counts or frequencies. The `fit()` method takes the training feature matrix `X_train` (produced by `CountVectorizer`) and the corresponding labels `y_train`, learning the probability distributions needed to classify new text samples. It essentially builds a statistical model of how likely each word is to appear in each class.

Making Predictions

```
python
y_pred = classifier.predict(X_test)
```

Once the model is trained, this line uses it to predict the labels for the test set. `X_test` contains the vectorized form of unseen text data, and `y_pred` stores the predicted class for each test sample. These predictions will be compared against the true labels to evaluate the model's performance.

Evaluating Model Performance

```
python
accuracy = accuracy_score(y_test, y_pred)
macro_precision = precision_score(y_test, y_pred, average='macro', zero_division=0)
macro_recall = recall_score(y_test, y_pred, average='macro', zero_division=0)
```

Here, you compute three key evaluation metrics:

- **Accuracy** measures the overall proportion of correct predictions.
- **Macro Precision** calculates precision for each class and then averages them equally, regardless of class size.
- **Macro Recall** does the same for recall. The `zero_division=0` argument prevents errors if a class has no predicted samples (i.e., avoids division by zero).

Displaying Results

```
python
print(f"Accuracy: {accuracy:.2f}")
print(f"Macro Precision: {macro_precision:.2f}")
print(f"Macro Recall: {macro_recall:.2f}")
print("\nPer-class metrics:")
print(classification_report(y_test, y_pred, zero_division=0))
```

This section prints the overall accuracy, macro-averaged precision and recall, and then uses `classification_report()` to show **per-class metrics**. This includes precision, recall, and F1-score for each label, helping you understand how well the model performs across different categories — especially useful in multi-class settings.

Confusion Matrix

```
python
cm = confusion_matrix(y_test, y_pred, labels=sorted(dataset['label'].unique()))
cm_df = pd.DataFrame(cm, index=sorted(dataset['label'].unique()),
columns=sorted(dataset['label'].unique()))
print("Confusion Matrix (rows=true, columns=predicted):")
print(cm_df)
```

The confusion matrix provides a **detailed breakdown of prediction outcomes**. Each row represents the actual class, and each column represents the predicted class. Diagonal values show correct predictions, while off-diagonal values indicate misclassifications. Wrapping it in a pandas DataFrame (`cm_df`) makes it easier to read and interpret, especially when dealing with multiple classes.

PROGRAM 2

```
import pandas as pd
import numpy as np
import math
from collections import defaultdict
from sklearn.model_selection import train_test_split

# Load dataset
data = pd.read_csv("naiv.csv")
print("First 5 rows of the dataset:")
print(data.head())

# Separate features and target
X = data.iloc[:, :-1].values # all columns except last
y = data.iloc[:, -1].values # last column

# Train/Test Split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, shuffle=True
)

# -----
# 3. Gaussian Naive Bayes (from scratch)
# -----
class GaussianNB_FromScratch:
    def fit(self, X, y):
        self.classes = np.unique(y)
        self.mean = {}
        self.var = {}
        self.priors = {}

        for c in self.classes:
            X_c = X[y == c]
            self.mean[c] = np.mean(X_c, axis=0)
            self.var[c] = np.var(X_c, axis=0) + 1e-6 # avoid zero division
            self.priors[c] = X_c.shape[0] / X.shape[0]

    def gaussian_pdf(self, class_idx, x):
        mean = self.mean[class_idx]
        var = self.var[class_idx]
        numerator = np.exp(-((x - mean) ** 2) / (2 * var))
        denominator = np.sqrt(2 * np.pi * var)
        return numerator / denominator

    def predict_one(self, x):
```

```

        posteriors = {}
        for c in self.classes:
            prior = np.log(self.priors[c])
            conditional = np.sum(np.log(self.gaussian_pdf(c, x)))
            posteriors[c] = prior + conditional
        return max(posteriors, key=posteriors.get)

    def predict(self, X):
        return np.array([self.predict_one(x) for x in X])

# -----
# 4. Train Model
# -----
model = GaussianNB_FromScratch()
model.fit(X_train, y_train)

# -----
# 5. Predictions
# -----
y_pred = model.predict(X_test)

# -----
# 6. Accuracy
# -----
accuracy = np.mean(y_pred == y_test)
print(f"\nAccuracy of Naive Bayes classifier: {accuracy * 100:.2f}%")

# -----
# 7. Show Predictions
# -----
comparison = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})
print("\nSample Predictions:")
print(comparison.head())

```



Loading the Dataset

```

python
data = pd.read_csv("naiv.csv")
print("First 5 rows of the dataset:")
print(data.head())

```

This block loads a CSV file named `naiv.csv` into a pandas DataFrame called `data`. The file is expected to contain structured tabular data, where each row represents a sample and each column represents a feature or label. The `head()` function displays the first five rows, giving you a quick preview to verify that the data has loaded correctly and to inspect its format.

Separating Features and Target

python

```
X = data.iloc[:, :-1].values
```

```
y = data.iloc[:, -1].values
```

Here, you're splitting the dataset into **features (X)** and **target labels (y)**. **X** includes all columns except the last one — these are the input variables used to make predictions. **y** is the last column, which contains the output or class label you're trying to predict. This separation is essential for supervised learning, where the model learns to map inputs to outputs.

Splitting into Training and Testing Sets

python

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, shuffle=True
)
```

This line uses scikit-learn's `train_test_split()` to divide the data into **training** and **testing** sets. 80% of the data is used for training (**X_train, y_train**), and 20% is reserved for testing (**X_test, y_test**). The `random_state=42` ensures reproducibility — meaning the same split will occur every time you run the code. `shuffle=True` randomizes the data before splitting, which helps prevent bias from any ordering in the original dataset.

```
class GaussianNB_FromScratch:
```

```
    def fit(self, X, y):
```

```
        self.classes = np.unique(y)
```

```
        self.mean = {}
```

```
        self.var = {}
```

```
        self.priors = {}
```

```
        for c in self.classes:
```

```
            X_c = X[y == c]
```

```
            self.mean[c] = np.mean(X_c, axis=0)
```

```
            self.var[c] = np.var(X_c, axis=0) + 1e-6 # avoid zero division
```

```
            self.priors[c] = X_c.shape[0] / X.shape[0]
```

```
    def gaussian_pdf(self, class_idx, x):
```

```
        mean = self.mean[class_idx]
```

```
        var = self.var[class_idx]
```

```
        numerator = np.exp(-(x - mean) ** 2) / (2 * var)
```

```
        denominator = np.sqrt(2 * np.pi * var)
```

```
        return numerator / denominator
```

```
    def predict_one(self, x):
```

```
        posteriors = {}
```

```

    for c in self.classes:
        prior = np.log(self.priors[c])
        conditional = np.sum(np.log(self.gaussian_pdf(c, x)))
        posteriors[c] = prior + conditional
    return max(posteriors, key=posteriors.get)

def predict(self, X):
    return np.array([self.predict_one(x) for x in X])

```

Class Definition: **GaussianNB_FromScratch**

This class encapsulates the logic for a **Gaussian Naive Bayes classifier**, which is a probabilistic model used for classification tasks. It assumes that features are **independent given the class** (the "naive" assumption) and that each feature follows a **normal (Gaussian) distribution** within each class. This model is especially useful for continuous-valued features.

fit(self, X, y): Training the Model

This method trains the classifier using the input features **X** and target labels **y**.

```
python
self.classes = np.unique(y)
```

Here, you're identifying all the **unique class labels** in the dataset. For example, if **y** contains values like **[0, 1, 0, 1, 1]**, this line extracts **[0, 1]**.

```
python
self.mean = {}
self.var = {}
self.priors = {}
```

These dictionaries will store:

- The **mean** of each feature for each class.
- The **variance** of each feature for each class.
- The **prior probability** of each class (i.e., how frequent each class is in the training data).

```
python
for c in self.classes:
    X_c = X[y == c]
```

This filters the dataset to include only the samples that belong to class `c`. You're essentially grouping the data by class.

```
python
self.mean[c] = np.mean(X_c, axis=0)
self.var[c] = np.var(X_c, axis=0) + 1e-6
```

For each class, you compute the **mean and variance** of each feature. These are the parameters of the Gaussian distribution for that class. The small constant `1e-6` is added to the variance to prevent division by zero during prediction — a common numerical safeguard.

```
python
self.priors[c] = X_c.shape[0] / X.shape[0]
```

This calculates the **prior probability** of class `c` — the proportion of samples in the dataset that belong to that class. It's used later to weigh the likelihoods during prediction.

`gaussian_pdf(self, class_idx, x): Likelihood Calculation`

This method computes the **probability density** of a feature vector `x` under the Gaussian distribution for a given class.

```
python
mean = self.mean[class_idx]
var = self.var[class_idx]
```

You retrieve the mean and variance for the specified class.

```
python
numerator = np.exp(-((x - mean) ** 2) / (2 * var))
denominator = np.sqrt(2 * np.pi * var)
return numerator / denominator
```

This is the formula for the **Gaussian (normal) probability density function** applied element-wise to each feature. It calculates how likely each feature value in `x` is, given the class-specific distribution. The result is a vector of probabilities — one for each feature.

`predict_one(self, x): Predicting a Single Sample`

This method computes the **posterior probability** for each class given a single input `x`, and returns the class with the highest score.

```
python
posteriors = {}
```

You initialize a dictionary to store the log-posterior probabilities for each class.

```
python
for c in self.classes:
    prior = np.log(self.priors[c])
    conditional = np.sum(np.log(self.gaussian_pdf(c, x)))
    posteriors[c] = prior + conditional
```

For each class:

- You compute the **log of the prior** (to avoid underflow and simplify multiplication into addition).
- You compute the **log of the likelihoods** for each feature and sum them — this gives the log of the joint likelihood.
- You add the prior and likelihood to get the **log-posterior**.

```
python
return max(posteriors, key=posteriors.get)
```

You return the class with the highest posterior probability — this is your prediction.

predict(self, X): Predicting Multiple Samples

This method applies `predict_one()` to every sample in the dataset `X`.

```
python
return np.array([self.predict_one(x) for x in X])
```

It loops through each sample `x` in `X`, predicts its class, and returns the results as a NumPy array. This makes the classifier usable on entire datasets, not just individual samples.

Model Initialization and Training

```
python
model = GaussianNB_FromScratch()
model.fit(X_train, y_train)
```

This block creates an instance of your custom `GaussianNB_FromScratch` class and trains it using the training data. The `fit()` method calculates the mean, variance, and prior probabilities for each class, assuming that each feature follows a Gaussian distribution. This step builds the statistical foundation the model will use to classify new samples.

Making Predictions

```
python
y_pred = model.predict(X_test)
```

Here, you use the trained model to predict the class labels for the test set. The `predict()` method applies `predict_one()` to each sample in `X_test`, computing the posterior probability for each class and selecting the one with the highest score. The result, `y_pred`, is a NumPy array of predicted labels.



Calculating Accuracy

```
python
accuracy = np.mean(y_pred == y_test)
print(f"\nAccuracy of Naive Bayes classifier: {accuracy * 100:.2f}%")
```

This line compares the predicted labels (`y_pred`) with the true labels (`y_test`) and calculates the **accuracy** — the proportion of correct predictions. Multiplying by 100 converts it to a percentage, and the formatted print statement displays the result with two decimal places. Accuracy gives you a quick sense of how well the model is performing overall.



Displaying Sample Predictions

```
python
comparison = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})
print("\nSample Predictions:")
print(comparison.head())
```

PROGRAM 1

```
import pandas as pd
import numpy as np
import math

def entropy(data):
    labels = data.iloc[:, -1]
    values, counts = np.unique(labels, return_counts=True)
    probabilities = counts / counts.sum()
    ent = -np.sum(probabilities * np.log2(probabilities))
    return ent

def information_gain(data, attr):
    total_entropy = entropy(data)
    values, counts = np.unique(data[attr], return_counts=True)
    weighted_entropy = 0.0
    for val, count in zip(values, counts):
        subset = data[data[attr] == val]
        subset_entropy = entropy(subset)
```

```

        weighted_entropy += (count / len(data)) * subset_entropy
    gain = total_entropy - weighted_entropy
    return gain

```

```

def id3(data, features, depth=0):
    labels = data.iloc[:, -1]
    if len(np.unique(labels)) == 1:
        return labels.iloc[0]
    if len(features) == 0:
        return labels.mode()[0]

    gains = {attr: information_gain(data, attr) for attr in features}
    best_attr = max(gains, key=gains.get)

    tree = {best_attr: {}}
    unique_vals = np.unique(data[best_attr])

    for val in unique_vals:
        subset = data[data[best_attr] == val]
        if subset.empty:
            tree[best_attr][val] = labels.mode()[0]
        else:
            subtree = id3(subset.drop(columns=[best_attr]), [f for f in features if f != best_attr],
                depth + 1)
            tree[best_attr][val] = subtree
    return tree

```

```

def print_tree(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + "Label:", tree)
        return
    for attr, branches in tree.items():
        for val, subtree in branches.items():
            print(f"{indent}{attr} = {val}:")
            print_tree(subtree, indent + " ")

```

```

def classify(tree, instance):
    """
    Classify a new instance using the decision tree.
    `instance` is a dict with attribute:value pairs.
    """
    if not isinstance(tree, dict):
        return tree # Leaf node label

    attr = next(iter(tree))
    val = instance.get(attr)

    # Handle unseen attribute value by returning None

```

```

    if val not in tree[attr]:
        return None

    subtree = tree[attr][val]
    return classify(subtree, instance)

def read_and_prepare_csv(csv_path):
    """
    Reads csv file and preprocesses it:
    - converts all columns to lowercase strings
    - returns dataframe
    """
    df = pd.read_csv(csv_path)
    print(df.head())
    for col in df.columns:
        df[col] = df[col].astype(str).str.lower()
    return df

if __name__ == "__main__":

    df = read_and_prepare_csv("weather.csv")

    # Exclude 'id' column from features explicitly
    features = [col for col in df.columns[:-1] if col != 'id']
    print("Information gain of each features")
    for feature in features:
        print(feature, information_gain(df, feature))

    tree = id3(df, features)

    print("Decision Tree:")
    print_tree(tree)

    new_instance = {'outlook': 'sunny', 'temperature': 'hot', 'humidity': 'normal', 'windy': 'True'}

    prediction = classify(tree, new_instance)
    print("\nClassify new instance:", new_instance)
    print("Predicted label:", prediction)
    id outlook temperature humidity windy play
0 1 sunny hot high False no
1 2 sunny hot high True no
2 3 overcast hot high False yes
3 4 rainy mild high False yes
4 5 rainy cool normal False yes
Information gain of each features
outlook 0.24674981977443933
temperature 0.02922256565895487
humidity 0.15183550136234159

```

windy 0.04812703040826949

Decision Tree:

outlook = overcast:

Label: yes

outlook = rainy:

windy = false:

Label: yes

windy = true:

Label: no

outlook = sunny:

humidity = high:

Label: no

humidity = normal:

Label: yes

Classify new instance: {'outlook': 'sunny', 'temperature': 'hot', 'humidity': 'normal', 'windy': 'True'}

Predicted label: yes

```
import pandas as pd
```

```
import numpy as np
```

```
import math
```

```
def entropy(data):
```

```
    labels = data.iloc[:, -1]
```

```
    values, counts = np.unique(labels, return_counts=True)
```

```
    probabilities = counts / counts.sum()
```

```
    ent = -np.sum(probabilities * np.log2(probabilities))
```

```
    return ent
```



Purpose of the Function

This function calculates the **entropy** of a dataset, which is a measure of **impurity or uncertainty** in the target variable. In the context of decision trees, entropy helps determine how “mixed” the class labels are in a given subset of data. Lower entropy means the data is more pure (mostly one class), while higher entropy means the data is more diverse (multiple classes evenly distributed). It’s a key concept in algorithms like ID3 and C4.5 for selecting the best feature to split on.



Step-by-Step Breakdown

```
python
```

```
labels = data.iloc[:, -1]
```

This line extracts the **last column** of the DataFrame, which is assumed to be the **target variable** (i.e., the class labels). This is what you'll analyze to compute entropy.

```
python
values, counts = np.unique(labels, return_counts=True)
```

Here, you use `np.unique()` to find all **unique class labels** and count how many times each one appears. For example, if the labels are `[yes, no, yes, yes, no]`, this returns `values = [no, yes]` and `counts = [2, 3]`.

```
python
probabilities = counts / counts.sum()
```

This converts the raw counts into **probabilities** — the relative frequency of each class. These probabilities are what you'll use to compute entropy.

```
python
ent = -np.sum(probabilities * np.log2(probabilities))
```

This is the **entropy formula**:

$$\text{Entropy} = - \sum_{i=1}^n p_i \log_2(p_i)$$

It multiplies each class probability by its log base 2, sums them up, and negates the result. The log base 2 is used because entropy is measured in **bits**. If all samples belong to one class, entropy is 0 (pure). If classes are evenly split, entropy is maximal (e.g., 1 for binary classes).

Return Value

```
python
return ent
```

The function returns the computed entropy value, which can be used to compare different splits in a decision tree. The goal is to choose splits that **minimize entropy**, leading to purer child nodes.

```
def information_gain(data, attr):
    total_entropy = entropy(data)
    values, counts = np.unique(data[attr], return_counts=True)
```

```

weighted_entropy = 0.0
for val, count in zip(values, counts):
    subset = data[data[attr] == val]
    subset_entropy = entropy(subset)
    weighted_entropy += (count / len(data)) * subset_entropy
gain = total_entropy - weighted_entropy
return gain

```

Purpose of the Function

The `information_gain` function calculates how much **predictive power** a given attribute provides about the target variable. It's a key metric used in decision tree algorithms like ID3 to decide which feature to split on at each node. The idea is simple: choose the attribute that **reduces uncertainty (entropy)** the most. The greater the reduction, the more informative the attribute.

Step-by-Step Breakdown

```

python
total_entropy = entropy(data)

```

This computes the **entropy of the entire dataset**, which represents the baseline uncertainty before any split. It reflects how mixed the class labels are across all samples.

```

python
values, counts = np.unique(data[attr], return_counts=True)

```

Here, you identify all **unique values** of the attribute `attr` and count how many times each value appears. For example, if `attr = 'weather'`, this might return `['sunny', 'rainy', 'overcast']` and their respective counts.

```

python
weighted_entropy = 0.0

```

You initialize a variable to accumulate the **weighted average entropy** of the subsets created by splitting on `attr`.

```

python
for val, count in zip(values, counts):
    subset = data[data[attr] == val]
    subset_entropy = entropy(subset)
    weighted_entropy += (count / len(data)) * subset_entropy

```

This loop iterates over each unique value of the attribute:

- It creates a **subset** of the data where `attr == val`.
- It computes the **entropy** of that subset.
- It multiplies the subset's entropy by its **relative size** (i.e., weight) and adds it to the total.

This gives you the expected entropy **after** splitting on the attribute — a measure of how uncertain the class labels remain.



Calculating Information Gain

python

```
gain = total_entropy - weighted_entropy
return gain
```

Finally, you subtract the weighted entropy from the original entropy to get the **information gain**. A higher gain means the attribute does a better job of reducing uncertainty, making it a strong candidate for splitting.

```
def id3(data, features, depth=0):
    labels = data.iloc[:, -1]
    if len(np.unique(labels)) == 1:
        return labels.iloc[0]
    if len(features) == 0:
        return labels.mode()[0]

    gains = {attr: information_gain(data, attr) for attr in features}
    best_attr = max(gains, key=gains.get)

    tree = {best_attr: {}}
    unique_vals = np.unique(data[best_attr])

    for val in unique_vals:
        subset = data[data[best_attr] == val]
        if subset.empty:
            tree[best_attr][val] = labels.mode()[0]
        else:
            subtree = id3(subset.drop(columns=[best_attr]), [f for f in features if f !=
best_attr], depth + 1)
            tree[best_attr][val] = subtree
    return tree
```



Purpose of the Function

The `id3` function builds a **decision tree** from a dataset using the **Iterative Dichotomiser 3 (ID3)** algorithm. It recursively selects the attribute that provides the highest **information gain**, splits the dataset based on that attribute's values, and continues building subtrees until it reaches a stopping condition. The result is a nested dictionary structure representing the decision tree.

Base Cases: When to Stop Recursion

```
python
labels = data.iloc[:, -1]
if len(np.unique(labels)) == 1:
    return labels.iloc[0]
```

This checks if all samples in the current subset belong to the same class. If so, the node is **pure**, and the function returns that class label — this becomes a **leaf node**.

```
python
if len(features) == 0:
    return labels.mode()[0]
```

If there are no more features left to split on, the function returns the **most frequent class label** in the current subset. This handles cases where the data is still mixed but no further splits are possible.

Selecting the Best Attribute

```
python
gains = {attr: information_gain(data, attr) for attr in features}
best_attr = max(gains, key=gains.get)
```

This block calculates the **information gain** for each remaining feature and selects the one with the **highest gain**. This attribute is considered the most informative and is chosen as the **splitting criterion** for the current node.

Building the Tree Structure

```
python
tree = {best_attr: {}}
unique_vals = np.unique(data[best_attr])
```

You initialize a dictionary to represent the current node in the tree, with `best_attr` as the key. Then, you find all **unique values** of that attribute — each value will lead to a different **branch** in the tree.

Recursively Building Subtrees

```
python
for val in unique_vals:
```

```

subset = data[data[best_attr] == val]
if subset.empty:
    tree[best_attr][val] = labels.mode()[0]
else:
    subtree = id3(subset.drop(columns=[best_attr]), [f for f in features if f !=
best_attr], depth + 1)
    tree[best_attr][val] = subtree

```

For each value of the best attribute:

- You create a **subset** of the data where the attribute equals that value.
- If the subset is empty (no samples match), you assign the **majority class** as a fallback.
- Otherwise, you **recursively call id3** on the subset, removing the used attribute from the feature list.
- The result is stored in the tree under the corresponding value.

This recursive process continues until all branches lead to leaf nodes.

Returning the Tree

```

python
return tree

```

Once all branches are built, the function returns the tree dictionary. This structure can be used for **classification**, **visualization**, or **interpretation** of decision paths.

```

def print_tree(tree, indent=""):
    if not isinstance(tree, dict):
        print(indent + "Label:", tree)
        return
    for attr, branches in tree.items():
        for val, subtree in branches.items():
            print(f"{indent}{attr} = {val}:")
            print_tree(subtree, indent + " ")

```

Purpose of the Function

The `print_tree` function provides a **textual visualization** of a decision tree represented as a nested dictionary. It helps you understand the structure of the tree — which attributes are used for splitting, what values lead to which branches, and where the leaf nodes (class labels) appear. This is especially useful for debugging, interpreting model logic, or presenting the tree in a human-readable format.

Base Case: Reaching a Leaf Node

```
python
if not isinstance(tree, dict):
    print(indent + "Label:", tree)
    return
```

This checks whether the current `tree` object is a dictionary. If it's not, that means you've reached a **leaf node** — a final prediction (class label). The function prints the label with the current indentation to show its depth in the tree, then returns. This prevents further recursion.

Recursive Case: Traversing the Tree

```
python
for attr, branches in tree.items():
    for val, subtree in branches.items():
        print(f'{indent}{attr} = {val}:')
        print_tree(subtree, indent + " ")
```

This block handles the **internal nodes** of the tree:

- `attr` is the attribute used for splitting at the current node.
- `branches` is a dictionary mapping each possible value of `attr` to a subtree.
- For each value `val`, the function prints the condition (e.g., `outlook = sunny:`) with the current indentation.
- It then recursively calls `print_tree` on the corresponding `subtree`, increasing the indentation to reflect the deeper level.

This recursive traversal continues until all branches are printed, resulting in a **hierarchical display** of the entire decision tree.

```
def classify(tree, instance):
    """
    Classify a new instance using the decision tree.
    `instance` is a dict with attribute:value pairs.
    """
    if not isinstance(tree, dict):
        return tree # Leaf node label

    attr = next(iter(tree))
    val = instance.get(attr)

    # Handle unseen attribute value by returning None
    if val not in tree[attr]:
        return None

    subtree = tree[attr][val]
```

```
return classify(subtree, instance)
```

Purpose of the Function

The `classify` function takes a **decision tree** (built using your `id3` algorithm) and a new **instance** (a dictionary of attribute-value pairs), and returns the predicted **class label**. It does this by **recursively walking through the tree**, following branches that match the instance's attribute values, until it reaches a leaf node.

Base Case: Reaching a Leaf Node

```
python
if not isinstance(tree, dict):
    return tree # Leaf node label
```

This checks whether the current `tree` object is a dictionary. If it's not, that means you've reached a **leaf node**, which contains a class label (e.g., `'yes'` or `'no'`). The function returns this label as the prediction. This is the stopping condition for the recursion.

Extracting the Splitting Attribute

```
python
attr = next(iter(tree))
val = instance.get(attr)
```

Here, you extract the **splitting attribute** at the current node — the key of the dictionary. For example, if the tree is `{outlook: {...}}`, then `attr = 'outlook'`. You then retrieve the value of this attribute from the input `instance`. This tells you which branch to follow next.

Handling Unseen Attribute Values

```
python
if val not in tree[attr]:
    return None
```

This line checks whether the instance's value for the current attribute exists in the tree's branches. If the value is **unseen** (i.e., not present in the training data), the function returns `None`. This is a safeguard against missing or novel input values that the tree doesn't know how to handle.

Recursively Traversing the Tree

```
python
subtree = tree[attr][val]
return classify(subtree, instance)
```

If the value is valid, you follow the corresponding branch to the **subtree**, and recursively call `classify()` on it. This continues until you reach a leaf node, at which point the class label is returned.

```
def read_and_prepare_csv(csv_path):
    """
    Reads csv file and preprocesses it:
    - converts all columns to lowercase strings
    - returns dataframe
    """
    df = pd.read_csv(csv_path)
    print(df.head())
    for col in df.columns:
        df[col] = df[col].astype(str).str.lower()
    return df

if __name__ == "__main__":

    df = read_and_prepare_csv("weather.csv")

    # Exclude 'id' column from features explicitly
    features = [col for col in df.columns[:-1] if col != 'id']
    print("Information gain of each features")
    for feature in features:
        print(feature, information_gain(df, feature))

    tree = id3(df, features)

    print("Decision Tree:")
    print_tree(tree)

    new_instance = {'outlook': 'sunny', 'temperature': 'hot', 'humidity': 'normal', 'windy':
    'True'}

    prediction = classify(tree, new_instance)
    print("\nClassify new instance:", new_instance)
    print("Predicted label:", prediction)
```



Reading and Preprocessing the CSV

```
python
def read_and_prepare_csv(csv_path):
    ...
```

This function reads a CSV file and converts all values in every column to lowercase strings. This preprocessing step ensures **consistency in data formatting**, which is crucial for string comparisons during tree construction and classification. For example, 'Sunny' and 'sunny' would otherwise be treated as different values. The function prints the first few rows for inspection and returns the cleaned DataFrame.

Main Execution Block

```
python
if __name__ == "__main__":
    df = read_and_prepare_csv("weather.csv")
```

This block runs when the script is executed directly. It loads and prepares the dataset from "weather.csv", which is assumed to contain categorical features and a target label in the last column. The data is now ready for feature selection and tree building.

Feature Selection and Information Gain

```
python
features = [col for col in df.columns[:-1] if col != 'id']
print("Information gain of each features")
for feature in features:
    print(feature, information_gain(df, feature))
```

You explicitly exclude the 'id' column from the feature list, assuming it's a non-informative identifier. Then, for each remaining feature, you calculate and print its **information gain** — a measure of how well the feature splits the data to reduce entropy. This helps you understand which features are most predictive of the target label.

Building the Decision Tree

```
python
tree = id3(df, features)
print("Decision Tree:")
print_tree(tree)
```

You build the decision tree using your custom `id3` function, which recursively selects the best feature to split on based on information gain. The resulting tree is a nested dictionary structure. You then print the tree using `print_tree`, which displays it in an indented, readable format showing how decisions are made at each node.

Classifying a New Instance

```
python
new_instance = {'outlook': 'sunny', 'temperature': 'hot', 'humidity': 'normal', 'windy': 'True'}
```

```
prediction = classify(tree, new_instance)
print("\nClassify new instance:", new_instance)
print("Predicted label:", prediction)
```

You define a new data point as a dictionary of attribute-value pairs. The `classify` function traverses the decision tree using these values, following branches that match the instance's attributes. If all values are recognized, it returns a predicted label (e.g., `'yes'` or `'no'`). This demonstrates how the tree can be used for real-time decision-making.